GITHUB REPO: https://github.com/enchi244/2025-CP_PAWFEEDS.git
RELEVANT CODE

1. Main Feeder Firmware (Hardware)

This is the Master.txt firmware file that integrates the main ESP32 controller and consists of methods for managing the physical feeder like, triggerDispense and handleRfidRead, and load cell (weight) sensing functions.

```
// Dispenser Management State
struct DispenserState {
  DispenserMode mode;
  Servo* servo;
  int bowlId;
  float targetTotalWeight;
  float lastStableWeight;
  unsigned long lastWeightChangeTime;
  unsigned long stateStartTime;
  int unclogAttempts;
  UnclogStep unclogStep;
};

// Core Dispense Method
void triggerDispense(int bowlNumber, int amount) {
  DispenserState* disp = (bowlNumber == 1) ? &dispenser1 : &dispenser2;
  HX711* scale = (bowlNumber == 1) ? &loadCellBowl1 : &loadCellBowl2;

  if (disp->mode != D_IDLE) {
    Serial.printf("[SERVO] Bowl %d is already busy. Command ignored.\n", bowlNumber);
```

```
  return;
 }

 float currentWeight = getWeightInGrams(*scale, bowlNumber);
 disp->targetTotalWeight = currentWeight + amount;
 disp->lastStableWeight = currentWeight;
 disp->lastWeightChangeTime = millis();
 disp->unclogAttempts = 0;
 disp->mode = D_DISPENSING;
 disp->servo->write(SERVO_FORWARD);
}
```

2. Camera Firmware (Hardware)

This is the Slave.txt camera firmware file that integrates the ESP32-CAM module and consists of methods for processing video feeds like, startCameraServer and stream_handler, and IP synchronization functions.

```
// Video Streaming Handler
esp_err_t stream_handler(httpd_req_t *req){
 camera_fb_t * fb = NULL;
 esp_err_t res = ESP_OK;
 size_t _jpg_buf_len = 0;
 uint8_t * _jpg_buf = NULL;
 char * part_buf[64];

 res = httpd_resp_set_type(req, _STREAM_CONTENT_TYPE);
```

```
  if (res != ESP_OK) return res;

  while (true) {
    fb = esp_camera_fb_get();
    if (!fb) {
      res = ESP_FAIL;
      break;
    }

    _jpg_buf_len = fb->len;
    _jpg_buf = fb->buf;

    // Send multipart jpeg frames
    if (res == ESP_OK) res = httpd_resp_send_chunk(req, _STREAM_BOUNDARY,
strlen(_STREAM_BOUNDARY));
    if (res == ESP_OK) {
      size_t hlen = snprintf((char *)part_buf, 64, _STREAM_PART, _jpg_buf_len);
      res = httpd_resp_send_chunk(req, (const char *)part_buf, hlen);
    }
    if (res == ESP_OK) res = httpd_resp_send_chunk(req, (const char *)_jpg_buf,
_jpg_buf_len);
    esp_camera_fb_return(fb);
    if (res != ESP_OK) break;

    delay(1); // 1ms delay for high FPS
  }
  return res;
```

```
}
```

3. Firebase Backend Functions (Software)

This is the index.ts backend service file that integrates the Firebase serverless environment and consists of methods for managing schedules and alerts like, scheduledFeedChecker and onFeederError, and push notification delivery functions.

```
// structure for Schedule data
interface ScheduleData {
 time: string;
 repeatDays: string[];
 petId: string;
 bowlNumber: number;
 portionGrams: number;
 isEnabled: boolean;
}

// Scheduled Feed Checker
export const scheduledFeedChecker = onSchedule(
 {
  schedule: "every 1 minutes",
  timeZone: "Asia/Singapore",
  region: "asia-southeast1",
 },
 async (event: ScheduledEvent) => {
  logger.info("Running scheduled feed checker...");
```

```
  // time parsing logic

  const feedersSnapshot = await firestore.collection("feeders").get();
  for (const feederDoc of feedersSnapshot.docs) {
   const schedulesRef = feederDoc.ref.collection("schedules");
   const q = schedulesRef.where("isEnabled", "==", true);

   q.get().then(async (scheduleSnapshot) => {
    for (const scheduleDoc of scheduleSnapshot.docs) {
     const schedule = scheduleDoc.data() as ScheduleData;

     if (schedule.time === currentTime && schedule.repeatDays.includes(currentDay)) {
      // Dispatch RTDB await_rfid command and trigger notification
     }
    }
   });
  }
 }
);
```

4. Video Relay Server (Software)

This is the relayserver_hls.js video relay file that integrates FFmpeg and consists of methods for processing live camera streams like, startStreamProcess, and RTMP network handling functions.

```
// Creates and starts a persistent ffmpeg
function startStreamProcess(streamKey, cameraUrl) {
```

```
  const outputUrl = `${RTMP_BASE_URL}/${streamKey}`;

  const ffmpegProcess = ffmpeg(cameraUrl)
    .inputFormat('mjpeg')
    .addInputOptions(FFMPEG_INPUT_OPTIONS)
    .videoFilters('setpts=PTS-STARTPTS') // Reset timestamps to 0 to prevent sync issues
    .addInputOption('-analyzeduration 500000')
    .addInputOption('-probesize 500000')
    .addOptions(FFMPEG_OUTPUT_OPTIONS)
    .output(outputUrl)
    .on('start', (commandLine) => {
      console.log(`[FFmpeg ${streamKey}] Active.`);
    })
    .on('error', (err, stdout, stderr) => {
      console.log(`[FFmpeg ${streamKey}] Reconnecting in 5 seconds...`);
      setTimeout(() => startStreamProcess(streamKey, cameraUrl), 5000);
    })
    .on('end', () => {
      setTimeout(() => startStreamProcess(streamKey, cameraUrl), 1000);
    });

  ffmpegProcess.run();
}
```

Master file

```
#include <Arduino.h>
```

```
#include <WiFi.h>
#include <Preferences.h>
#include <Firebase_ESP_Client.h>
#include <ESP32Servo.h>
#include <time.h>
#include <HX711.h>
#include <HardwareSerial.h>
#include <HTTPClient.h>
#include <ArduinoJson.h>

// ========== USER CONFIGURATION ==========
#define FIREBASE_PROJECT_ID "pawfeeds-v2"
#define FIREBASE_DATABASE_URL "https://pawfeeds-v2-default-rtdb.asia-southeast1.firebasedatabase.app/"
#define SERVICE_ACCOUNT_CLIENT_EMAIL "firebase-adminsdk-fbsvc@pawfeeds-v2.iam.gserviceaccount.com"
#define SERVICE_ACCOUNT_PRIVATE_KEY "-----BEGIN PRIVATE KEY-----\n" \
"MIIEvwIBADANBgkqhkiG9w0BAQEFAASCBKkwggSlAgEAAoIBAQC+Sqt8BKfCEgbp\n" \
"/q4XmzzpsKwlsKJesbWN/Sp2PO7nVgPqwEz9YeCy/aI98IjyGY5hujaLLNgB0GYE\n" \
"NLvWkAf3T6VQPUa8SB28XU5sT7tfCJu/1F3yYYAVHHpbg2gbGxjgPWVlQ5arww57\n" \
"HHREzOUTRTO5k9hRAzzf1kdjBJbboewEEclQH2uAdn+B9L58SOezjVONpr63+R5V\n" \
"WbZ7+B64BFE62Xsjcmk49OmHjKc+2ID80S6EWx6rxM5jGdTsUVtA0RISG5r1qeB/\n" \
"DazLCx1jexutuMHTogvAIirHGIMdOCW12USD8b+CRvP7Lsj6CwR91JSVm/b5P1pI\n" \
"0MNAgkfrAgMBAAECggEAO5iNHIki5P/aVHxjr5b5u8KOF3u7TmbfkmmAW+I3dNIW\n" \
"hfXV5uE5izUuE7H6YcApPGgiXvIbcG4BFT4iue7/3698+aVHOv5m+bBLOFa8OuYq\n" \
"SSjMh3WLtJDnrTN5bkvNPaVc1RsW3BJJvbrKmyWEdMWOjodEDxMxhHTKhLNSP9Rq\n" \
"cTz85kc4/IsXuAD6fmApysrWiVuS+vclzFGCjqQDozojsd58KW6XFa8PEZ95bUbE\n" \
"/Q8ZAigB88un8+87NpLd/usUk2FWHNJz523AJUA4qQE0N8J6E/7+dJbRJ9Ucg6lk\n" \
"kO0fRiLO7c7o6/7bW1iy4Yq+ph5N3+vc2pYzVZlkAQKBgQD0AbH86UoJxroGh/id\n" \
```

```
"849814dnEGugSQ1C2XCPNpQy19WtyfZvMM5yphfxfxxohJwr+KyQBcfBatGo/ipV\n" \
"9clJUOalKa+q9KzxLsSka41cvu4RsnVjWyVqGS4w/BdYFdTvNkuK3cGiK8qYwBaW\n" \
"i50PNrcCQ+7gmMCjkPJ6LDOh6wKBgQDHpRYRXk9ipsOw4KHBxn4rph8G6ooUIvzs\n" \
"Pie0e+A+pz9grNDC7y0a+k8fA4eq/iJPmu+FvuJW4fahCkt98VuW0EJjcqtBNgCB\n" \
"YbOKn9H+V4WaQKCHYyhl/QxGio4qm4z34cEVt/kMGwGHlaX39RILc3kEq2RfTSc7\n" \
"2FikgZxyAQKBgQCz61wWpN5W/xXEIxaLQUCYSUQqFs2FTthcZoC82P3Fz6hbkQQJ\n" \
"UO+pUhdtltCXsNCHC8ISIHD+iYk3FtKYt7HvtJudRXOmluu+m0GcC0IdFRvuKKyu\n" \
"KlMYPKD2tatw5AgyqtJg/sr8jVXB9EGzmBajVTD0lqrZKUlCUmq480bPKQKBgQCp\n" \
"To0qaYp1JOur4rQK+uQg7B4/5UL31LwdNJDDdJI1T+xldekMh3z+9euHZ5z0G9TJ\n" \
"IaGjEMAt4i8fXvWqdravbSn/4EzvXnaLQmnaU7LoORzqNYhtiF/ILhLs96+c3pFr\n" \
"3h2652vjljvn2bclUuLcpy6oERlr4Kg3DkAOMoSUAQKBgQCkTg6e4Ndg1d2SySFQ\n" \
"4h7YHQZ3larxK/03DFoElNIf/6zgiNL13+9nM3l4K8YAEG1OEcn5JaBdktmGrEWR\n" \
"A+zV7Qp/hZSVuOM4sxpGa7q94A5253tvLvegIvzhFYW4WQrVFadI5xZ1HaO2Ad9I\n" \
"Kzz19+jbd085r+kyTM5X0UVzZA==\n" \
"-----END PRIVATE KEY-----\n"

#define SERVO_BOWL_1_PIN 21
#define SERVO_BOWL_2_PIN 22
#define HX711_BOWL1_DOUT_PIN 27
#define HX711_BOWL1_SCK_PIN 26
#define HX711_BOWL2_DOUT_PIN 19
#define HX711_BOWL2_SCK_PIN 18

// --- RFID PINS ---
#define RFID1_RX_PIN 16
#define RFID1_TX_PIN 17
#define RFID2_RX_PIN 4
#define RFID2_TX_PIN 2
```

```
#define SLAVE_1_SERIAL_TX_PIN 33
#define SLAVE_1_SERIAL_RX_PIN 32
#define SLAVE_2_SERIAL_TX_PIN 15
#define SLAVE_2_SERIAL_RX_PIN 14

#define SERVO_FORWARD 0
#define SERVO_STOP 90
#define SERVO_REVERSE 180
#define STALL_TIMEOUT_MS 10000
#define MIN_WEIGHT_CHANGE_THRESHOLD 1.0
#define MAX_UNCLOG_ATTEMPTS 3

#define MAX_WIFI_RETRIES 3

const char* AP_SSID = "PawFeeds_Setup";
const char* AP_PASSWORD = NULL;

#define CALIBRATION_FACTOR_1 -450.0
#define CALIBRATION_FACTOR_2 -450.0

// --- Global Objects ---
WiFiServer server(80);
Preferences preferences;
FirebaseData stream_fbdo;
FirebaseData fbdo;
FirebaseAuth auth;
FirebaseConfig config;
Servo servoBowl1;
Servo servoBowl2;
```

```
HX711 loadCellBowl1;
HX711 loadCellBowl2;
HardwareSerial SerialRFID1(1);
HardwareSerial SerialRFID2(2);

String feederId = "";
String streamPath = "/commands/";
bool streamActive = false;
unsigned long lastCmdTimestamp = 0;
unsigned long lastWeightUpdateTime = 0;
int wifiConnectionRetries = 0;

// --- HUNGER DETECTION COUNTERS ---
int unscheduledScansBowl1 = 0;
int unscheduledScansBowl2 = 0;

// --- MULTI-BOWL STATE MANAGEMENT (Scheduling) ---
struct BowlWaitState {
  bool active;
  String expectedTag;
  int amount;
  String scheduleId;
  String petId;
  unsigned long waitStartTime;
};

BowlWaitState bowl1State = {false, "", 0, "", "", 0};
BowlWaitState bowl2State = {false, "", 0, "", "", 0};
```

```
// --- CLOSED-LOOP DISPENSER MANAGEMENT ---
enum DispenserMode {
  D_IDLE,
  D_DISPENSING,
  D_UNCLOGGING
};

// --- Unclog Sequence Steps ---
enum UnclogStep {
  U_STEP_1_REVERSE,
  U_STEP_2_FORWARD,
  U_STEP_3_REVERSE,
  U_STEP_4_FORWARD,
  U_COMPLETE
};

struct DispenserState {
  DispenserMode mode;
  Servo* servo;
  int bowlId;
  float targetTotalWeight;
  float lastStableWeight;
  unsigned long lastWeightChangeTime;
  unsigned long stateStartTime;
  int unclogAttempts;
  UnclogStep unclogStep;
};

DispenserState dispenser1 = {D_IDLE, &servoBowl1, 1, 0, 0, 0, 0, 0, U_STEP_1_REVERSE};
```

```
DispenserState dispenser2 = {D_IDLE, &servoBowl2, 2, 0, 0, 0, 0, 0, U_STEP_1_REVERSE};

// --- Global Modes ---
enum GlobalMode { NORMAL, PAIRING };
GlobalMode globalMode = NORMAL;
int pairingBowlTarget = 0;
unsigned long pairingStartTime = 0;

// --- Buffers ---
String rfid1_buffer = "";
String rfid2_buffer = "";
unsigned long lastRfid1ByteTime = 0;
unsigned long lastRfid2ByteTime = 0;
const long rfidTagTimeout = 50;
String lastScannedTag = "";
unsigned long lastLogTime = 0;

volatile bool newCommandAvailable = false;
volatile bool clearCommandNode = false;
struct Command {
  int bowl;
  int amount;
};
Command pendingCommand;

// --- Status Update Queue ---
struct StatusUpdatePacket {
  int bowl;
  bool isWaiting;
```

```
  int amount;
  String petId;
};
volatile bool statusUpdateAvailable = false;
StatusUpdatePacket statusUpdateQueue;

enum DeviceState {
  PROVISIONING_MODE,
  CONNECTING_WIFI,
  SYNCING_TIME,
  SYNCING_TIME_HTTP_FALLBACK,
  AUTHENTICATING_FIREBASE,
  REGISTERING_FIREBASE,
  OPERATIONAL,
  RESETTING,
  ERROR
};
DeviceState currentState = PROVISIONING_MODE;

// --- Function Prototypes ---
void tokenStatusCallback(TokenInfo info);
void connectToWiFi();
void syncTime();
void syncTimeWithHttpFallback();
void authenticateWithFirebase();
void registerDeviceWithFirestore();
void startProvisioningServer();
void handleClient();
String urlDecode(String str);
```

```
String parseFeederIdFromResponse(String response);
void triggerDispense(int bowl, int amount);
void handleDispensers();
void processPendingCommand();
void startRTDBStream();
void streamCallback(FirebaseStream data);
void streamTimeoutCallback(bool timeout);
void forwardSlaveLogs();
bool sendToSlave(HardwareSerial& serial, const String& packet);
void forwardPacketToSlaves(const String& packet, const String& packetType);
void updateFoodLevelsInFirestore();
float getWeightInGrams(HX711& sensor, int sensorNum);
void updateRfidBuffers();
void handleRfidRead(int readerNum, String scannedTagId);
void checkRfidTimeouts();
void updateBowlStatus(int bowl, bool isWaiting, int amount, String petId);
void sendEmptyErrorNotification(int bowlId);
void sendHungerAlert(int bowlId, String tagId);

void forwardPacketToSlaves(const String& packet, const String& packetType) {
  Serial.printf("[PROVISIONING] Forwarding %s to slave cameras...\n", packetType.c_str());
  SerialRFID1.end();
  SerialRFID2.end();
  delay(50);

  Serial.println("[PROVISIONING] Contacting Slave 1...");
  Serial2.begin(9600, SERIAL_8N1, SLAVE_1_SERIAL_RX_PIN, SLAVE_1_SERIAL_TX_PIN);
  if (sendToSlave(Serial2, packet)) {
    Serial.printf("[PROVISIONING] %s sent to Slave 1 successfully.\n", packetType.c_str());
```

```
  } else {
    Serial.printf("[ERROR] Failed to send %s to Slave 1.\n", packetType.c_str());
  }
  Serial2.end();
  delay(200);

  Serial.println("[PROVISIONING] Contacting Slave 2...");
  Serial1.begin(9600, SERIAL_8N1, SLAVE_2_SERIAL_RX_PIN, SLAVE_2_SERIAL_TX_PIN);
  if (sendToSlave(Serial1, packet)) {
    Serial.printf("[PROVISIONING] %s sent to Slave 2 successfully.\n", packetType.c_str());
  } else {
    Serial.printf("[ERROR] Failed to send %s to Slave 2.\n", packetType.c_str());
  }
  Serial1.end();

  Serial.println("[PROVISIONING] Forwarding complete.");

  SerialRFID1.begin(9600, SERIAL_8N1, RFID1_RX_PIN, RFID1_TX_PIN);
  SerialRFID2.begin(9600, SERIAL_8N1, RFID2_RX_PIN, RFID2_TX_PIN);
}

void setup() {
  Serial.begin(115200);
  Serial.println("\n[PawFeeds] Booting (Robust WiFi Logic)...");
  servoBowl1.attach(SERVO_BOWL_1_PIN);
  servoBowl2.attach(SERVO_BOWL_2_PIN);

  servoBowl1.write(SERVO_STOP);
  servoBowl2.write(SERVO_STOP);
```

```
  Serial.println("[HX711] Initializing weight sensor 1 (BOWL SCALE)...");
  loadCellBowl1.begin(HX711_BOWL1_DOUT_PIN, HX711_BOWL1_SCK_PIN);
  loadCellBowl1.set_scale(CALIBRATION_FACTOR_1);
  loadCellBowl1.tare();

  Serial.println("[HX711] Initializing weight sensor 2 (BOWL SCALE)...");
  loadCellBowl2.begin(HX711_BOWL2_DOUT_PIN, HX711_BOWL2_SCK_PIN);
  loadCellBowl2.set_scale(CALIBRATION_FACTOR_2);
  loadCellBowl2.tare();
  Serial.println("[HX711] Ready.");

  delay(5000);

  Serial.println("[RFID] Initializing RFID reader 1 (Bowl 1)...");
  SerialRFID1.begin(9600, SERIAL_8N1, RFID1_RX_PIN, RFID1_TX_PIN);
  Serial.println("[RFID] Initializing RFID reader 2 (Bowl 2)...");
  SerialRFID2.begin(9600, SERIAL_8N1, RFID2_RX_PIN, RFID2_TX_PIN);

  preferences.begin("pawfeeds", false);

  if (preferences.getString("ssid", "").length() > 0) {
    currentState = CONNECTING_WIFI;
  } else {
    currentState = PROVISIONING_MODE;
    startProvisioningServer();
  }
}
```

```
void loop() {
  forwardSlaveLogs();

  if (currentState == OPERATIONAL) {
    updateRfidBuffers();
    checkRfidTimeouts();
    handleDispensers();
  }

  switch (currentState) {
    case PROVISIONING_MODE:
      handleClient();
      break;
    case CONNECTING_WIFI:
      connectToWiFi();
      break;
    case SYNCING_TIME:
      syncTime();
      break;
    case SYNCING_TIME_HTTP_FALLBACK:
      syncTimeWithHttpFallback();
      break;
    case AUTHENTICATING_FIREBASE:
      authenticateWithFirebase();
      break;
    case REGISTERING_FIREBASE:
      registerDeviceWithFirestore();
      break;
    case OPERATIONAL:
```

```
  if (!streamActive) {
    startRTDBStream();
  }
  if (Firebase.ready() && !Firebase.RTDB.readStream(&stream_fbdo)) {
    Serial.println("[ERROR] Stream read error!");
    streamActive = false;
  }

  if (statusUpdateAvailable) {
    updateBowlStatus(statusUpdateQueue.bowl, statusUpdateQueue.isWaiting,
statusUpdateQueue.amount, statusUpdateQueue.petId);
    statusUpdateAvailable = false;
  }

  if (newCommandAvailable) {
    processPendingCommand();
    newCommandAvailable = false;
  }

  if (clearCommandNode) {
    Serial.println("[SYSTEM] Clearing command node from main loop.");
    String fullStreamPath = streamPath;
    fullStreamPath.concat(feederId);

    fbdo.clear();
    if (Firebase.RTDB.setString(&fbdo, fullStreamPath.c_str(), "null")) {
      Serial.println("[SYSTEM] Command node cleared successfully.");
    } else {
```

```
        Serial.printf("[SYSTEM] FAILED to clear command node: %s\n",
fbdo.errorReason().c_str());
      }
      clearCommandNode = false;
    }

    if (millis() - lastWeightUpdateTime > 300000) {
      Serial.println("[SYSTEM] Updating weight readings...");
      updateFoodLevelsInFirestore();
      lastWeightUpdateTime = millis();
    }
    break;
  case RESETTING:
    streamActive = false;
    Serial.println("[RESET] Entered RESETTING state.");
    forwardPacketToSlaves("RESET_DEVICE\n", "RESET COMMAND");
    Serial.println("[RESET] Wiping Master credentials and rebooting...");
    preferences.clear();
    ESP.restart();
    break;

  case ERROR:
    Serial.println("[ERROR] Error state reached. Restarting system in 10 seconds...");
    delay(10000);
    ESP.restart();
    break;
 }
 delay(10);
}
```

```
void tokenStatusCallback(TokenInfo info) {
  if (info.status == token_status_ready) {
    Serial.println("[AUTH] Token obtained successfully.");
  } else if (info.status == token_status_error) {
    Serial.printf("[AUTH] Token error: %s\n", info.error.message.c_str());
  }
}

// --- WIFI LOGIC ---

void connectToWiFi() {
  String savedSSID = preferences.getString("ssid");
  String savedPass = preferences.getString("pass");

  Serial.printf("[WIFI] Connecting to %s (Attempt %d/%d)...\n", savedSSID.c_str(),
wifiConnectionRetries + 1, MAX_WIFI_RETRIES);

  WiFi.begin(savedSSID.c_str(), savedPass.c_str());

  int attempts = 0;
  while (WiFi.status() != WL_CONNECTED && attempts < 30) {
    delay(500);
    Serial.print(".");
    attempts++;
  }

  if (WiFi.status() == WL_CONNECTED) {
    Serial.println("\n[WIFI] Connected!");
```

```
    wifiConnectionRetries = 0;
    currentState = SYNCING_TIME;
  } else {
    Serial.println("\n[WIFI] Failed.");
    wifiConnectionRetries++;

    if (wifiConnectionRetries >= MAX_WIFI_RETRIES) {
      Serial.println("\n[WIFI] CRITICAL: Maximum retries reached!");
      Serial.println("[WIFI] Fallback triggered: Wiping credentials and starting AP Mode.");

      preferences.clear();
      WiFi.disconnect(true);
      delay(1000);

      startProvisioningServer();

      wifiConnectionRetries = 0;
      currentState = PROVISIONING_MODE;
    } else {
      Serial.println("[WIFI] Retrying in 2 seconds...");
      delay(2000);
    }
  }
}

void syncTime() {
  Serial.println("[TIME] Syncing with NTP server...");
  configTime(8 * 3600, 0, "pool.ntp.org", "time.nist.gov", "time.google.com");
  time_t now = time(nullptr);
```

```
  int attempts = 0;

  while (now < 8 * 3600 * 2 && attempts < 30) {
    delay(500);
    now = time(nullptr);
    attempts++;
  }

  if (now < 8 * 3600 * 2) {
    Serial.println("[TIME] Failed to sync time via NTP!");
    Serial.println("[TIME] Attempting HTTP time fallback...");
    currentState = SYNCING_TIME_HTTP_FALLBACK;
  } else {
    struct tm timeinfo;
    gmtime_r(&now, &timeinfo);
    Serial.print("[TIME] Time synced successfully via NTP: ");
    Serial.print(asctime(&timeinfo));
    currentState = AUTHENTICATING_FIREBASE;
  }
}

void syncTimeWithHttpFallback() {
  Serial.println("[TIME_HTTP] Fetching time from World Time API...");
  HTTPClient http;

  if (!http.begin("http://worldtimeapi.org/api/ip")) {
    Serial.println("[TIME_HTTP] Failed to begin HTTP client. Retrying NTP...");
    delay(3000);
    currentState = SYNCING_TIME;
```

```
  return;
}

int httpCode = http.GET();
if (httpCode > 0) {
  if (httpCode == HTTP_CODE_OK) {
    String payload = http.getString();
    DynamicJsonDocument doc(1024);
    DeserializationError error = deserializeJson(doc, payload);

    if (error) {
      Serial.print(F("[TIME_HTTP] deserializeJson() failed: "));
      Serial.println(error.f_str());
      delay(3000);
      currentState = SYNCING_TIME;
    } else {
      if (doc.containsKey("unixtime")) {
        time_t now = doc["unixtime"];
        struct timeval tv;
        tv.tv_sec = now;
        tv.tv_usec = 0;
        settimeofday(&tv, NULL);

        struct tm timeinfo;
        gmtime_r(&now, &timeinfo);
        Serial.print("[TIME_HTTP] Time synced successfully via HTTP: ");
        Serial.print(asctime(&timeinfo));
        currentState = AUTHENTICATING_FIREBASE;
      } else {
```

```
        Serial.println("[TIME_HTTP] 'unixtime' field not found. Retrying NTP...");
        delay(3000);
        currentState = SYNCING_TIME;
       }
      }
    } else {
      Serial.printf("[TIME_HTTP] GET request failed, error: %s. Retrying NTP...\n",
http.errorToString(httpCode).c_str());
      delay(3000);
      currentState = SYNCING_TIME;
    }
  } else {
    Serial.printf("[TIME_HTTP] GET request failed, error: %s. Retrying NTP...\n",
http.errorToString(httpCode).c_str());
    delay(3000);
    currentState = SYNCING_TIME;
  }
  http.end();
}

void authenticateWithFirebase() {
  static bool config_initiated = false;
  static unsigned long authStartTime = 0;

  if (Firebase.ready()) {
    Serial.println("[AUTH] Authenticated!");
    feederId = preferences.getString("feederId", "");
    if (feederId.length() > 0) {
      currentState = OPERATIONAL;
```

```
  } else {
    currentState = REGISTERING_FIREBASE;
  }
  config_initiated = false;
  return;
}

if (!config_initiated) {
  Serial.println("[AUTH] Configuring service account (One-time init)...");
  config.database_url = FIREBASE_DATABASE_URL;
  config.service_account.data.client_email = SERVICE_ACCOUNT_CLIENT_EMAIL;
  config.service_account.data.project_id = FIREBASE_PROJECT_ID;
  config.service_account.data.private_key = SERVICE_ACCOUNT_PRIVATE_KEY;
  config.token_status_callback = tokenStatusCallback;

  stream_fbdo.setBSSLBufferSize(4096, 1024);
  fbdo.setBSSLBufferSize(2048, 1024);

  config.timeout.socketConnection = 60000;
  config.timeout.serverResponse = 40000;
  config.timeout.wifiReconnect = 10000;

  Firebase.reconnectWiFi(true);
  Serial.printf("[AUTH] Free Heap Before Begin: %d\n", ESP.getFreeHeap());
  Firebase.begin(&config, &auth);

  config_initiated = true;
  authStartTime = millis();
  Serial.println("[AUTH] Firebase.begin() called. Waiting for token...");
```

```
  } else {
    if (millis() - authStartTime > 60000) {
      Serial.println("[AUTH] TIMEOUT waiting for token! Resetting ESP to clear SSL stack.");
      delay(1000);
      ESP.restart();
    }

    static unsigned long lastPrint = 0;
    if (millis() - lastPrint > 1000) {
      Serial.print(".");
      lastPrint = millis();
    }
  }
}

void registerDeviceWithFirestore() {
  if (!Firebase.ready()) {
    Serial.println("[FIREBASE] Waiting for auth token...");
    return;
  }

  Serial.println("[FIREBASE] Registering device...");
  String owner_uid = preferences.getString("owner_uid", "");

  if (owner_uid.length() == 0) {
    Serial.println("[ERROR] No Owner UID found. Returning to Provisioning.");
    currentState = PROVISIONING_MODE;
    startProvisioningServer();
    return;
```

```
  }

  FirebaseJson content;
  content.set("fields/owner_uid/stringValue", owner_uid);
  content.set("fields/online/booleanValue", true);

  float initialWeight1 = getWeightInGrams(loadCellBowl1, 1);
  float initialWeight2 = getWeightInGrams(loadCellBowl2, 2);
  int initialWeight1_int = (int)initialWeight1;
  int initialWeight2_int = (int)initialWeight2;

  content.set("fields/foodLevels/mapValue/fields/1/integerValue", initialWeight1_int);
  content.set("fields/foodLevels/mapValue/fields/2/integerValue", initialWeight2_int);

  String feederContent;
  content.toString(feederContent, false);

  fbdo.clear();

  if (Firebase.Firestore.createDocument(&fbdo, FIREBASE_PROJECT_ID, "", "feeders",
feederContent.c_str())) {
    Serial.println("[FIREBASE] Feeder document created.");
    feederId = parseFeederIdFromResponse(fbdo.payload());

    if (feederId.length() > 0) {
      preferences.putString("feederId", feederId);
      String feederIdPacket = "FEEDER_ID|";
      feederIdPacket.concat(feederId);
      feederIdPacket.concat("\n");
```

```
    forwardPacketToSlaves(feederIdPacket, "Feeder ID");

    String usersPath = "users/";
    usersPath.concat(owner_uid);

    FirebaseJson userUpdateContent;
    userUpdateContent.set("fields/feederId/stringValue", feederId);
    String userContent;
    userUpdateContent.toString(userContent, false);

    fbdo.clear();

    if (Firebase.Firestore.patchDocument(&fbdo, FIREBASE_PROJECT_ID, "",
usersPath.c_str(), userContent.c_str(), "feederId")) {
      Serial.println("[FIREBASE] User patched. Setup complete!");
      String setupSyncPath = "/setup_sync/";
      setupSyncPath.concat(owner_uid);

      FirebaseJson syncJson;
      syncJson.set("status", "complete");
      syncJson.set("feederId", feederId);

      fbdo.clear();
      if (Firebase.RTDB.setJSON(&fbdo, setupSyncPath.c_str(), &syncJson)) {
        Serial.println("[RTDB] Setup completion signal sent to App.");
      } else {
        Serial.printf("[RTDB] Failed to send setup signal: %s\n", fbdo.errorReason().c_str());
      }
```

```
        currentState = OPERATIONAL;
      } else {
        Serial.printf("[ERROR] Failed to patch user: %s. Retrying...\n", fbdo.errorReason().c_str());
        delay(2000);
      }
    } else {
      Serial.println("[ERROR] Failed to parse feeder ID. Retrying...");
      delay(2000);
    }
  } else {
    Serial.printf("[ERROR] Failed to create feeder doc: %s. Retrying...\n",
fbdo.errorReason().c_str());
    delay(2000);
  }
}

void startProvisioningServer() {
  Serial.print("[PROVISIONING] Starting Access Point: ");
  Serial.println(AP_SSID);
  WiFi.softAP(AP_SSID, AP_PASSWORD);
  IPAddress IP = WiFi.softAPIP();
  Serial.print("[PROVISIONING] AP IP address: ");
  Serial.println(IP.toString());
  server.begin();
  Serial.println("[PROVISIONING] Web server started.");
}

void handleClient() {
  WiFiClient client = server.available();
```

```
if (!client) return;

String currentLine = "";
unsigned long timeout = millis();
String header = "";

while (client.connected() && millis() - timeout < 2000) {
  if (client.available()) {
    char c = client.read();
    header += c;
    if (c == '\n') {
      if (currentLine.length() == 0) {
        if (header.indexOf("GET /networks") >= 0) {
          int n = WiFi.scanNetworks();
          String json = "[";
          for (int i = 0; i < n; ++i) {
            if (i > 0) json.concat(",");
            json.concat("{\"ssid\":\"");
            json.concat(WiFi.SSID(i));
            json.concat("\",\"rssi\":");
            json.concat(WiFi.RSSI(i));
            json.concat("}");
          }
          json += "]";
          client.println("HTTP/1.1 200 OK");
          client.println("Content-type:application/json");
          client.println("Connection: close");
          client.println();
          client.println(json);
```

```
    } else if (header.indexOf("POST /save") >= 0) {
      String body;
      while (client.available()) { body += (char)client.read(); }

      int ssid_start = body.indexOf("ssid=") + 5;
      int ssid_end = body.indexOf("&", ssid_start);
      String ssid = urlDecode(body.substring(ssid_start, ssid_end));

      int pass_start = body.indexOf("pass=") + 5;
      int pass_end = body.indexOf("&", pass_start);
      String pass = urlDecode(body.substring(pass_start, pass_end));

      int uid_start = body.indexOf("uid=") + 4;
      String uid = urlDecode(body.substring(uid_start));

      preferences.putString("ssid", ssid);
      preferences.putString("pass", pass);
      preferences.putString("owner_uid", uid);

      String credentialsPacket = ssid;
      credentialsPacket.concat("|");
      credentialsPacket.concat(pass);
      credentialsPacket.concat("\n");
      forwardPacketToSlaves(credentialsPacket, "Credentials");

      client.println("HTTP/1.1 200 OK\r\nContent-type:text/html\r\n\r\n<h1>Saved!
Restarting...</h1>");
      delay(1500);
    }
```

```
          break;
        } else {
          currentLine = "";
        }
      } else if (c != '\r') {
        currentLine += c;
      }
    }
  }
  client.stop();
  if (header.indexOf("POST /save") >= 0) {
    Serial.println("[PROVISIONING] Credentials saved. Restarting device.");
    ESP.restart();
  }
}

String parseFeederIdFromResponse(String response) {
  FirebaseJson json;
  json.setJsonData(response);
  FirebaseJsonData result;

  if (json.get(result, "name")) {
    String path = result.to<String>();
    int lastSlash = path.lastIndexOf('/');
    if (lastSlash != -1) {
      return path.substring(lastSlash + 1);
    }
  }
  return "";
```

```
}

String urlDecode(String str) {
  String decodedString = "";
  char temp[] = "0x00";
  char c;

  for (size_t i = 0; i < str.length(); i++) {
    c = str.charAt(i);
    if (c == '+') {
      decodedString += ' ';
    } else if (c == '%') {
      i++;
      temp[2] = str.charAt(i);
      i++;
      temp[3] = str.charAt(i);
      decodedString += (char)strtol(temp, NULL, 16);
    } else {
      decodedString += c;
    }
  }
  return decodedString;
}

void processPendingCommand() {
  if (pendingCommand.amount > 0 && pendingCommand.bowl > 0) {
    Serial.printf("[COMMAND] Processing FEED NOW command! Bowl: %d, Amount: %d\n",
pendingCommand.bowl, pendingCommand.amount);
    triggerDispense(pendingCommand.bowl, pendingCommand.amount);
```

```
    clearCommandNode = true;
  }
}

// --- CLOSED-LOOP DISPENSER LOGIC ---

void triggerDispense(int bowlNumber, int amount) {
  DispenserState* disp = (bowlNumber == 1) ? &dispenser1 : &dispenser2;
  HX711* scale = (bowlNumber == 1) ? &loadCellBowl1 : &loadCellBowl2;

  if (disp->mode != D_IDLE) {
    Serial.printf("[SERVO] Bowl %d is already busy. Command ignored.\n", bowlNumber);
    return;
  }

  float currentWeight = getWeightInGrams(*scale, bowlNumber);
  disp->targetTotalWeight = currentWeight + amount;

  disp->lastStableWeight = currentWeight;
  disp->lastWeightChangeTime = millis();
  disp->unclogAttempts = 0;
  disp->mode = D_DISPENSING;

  disp->servo->write(SERVO_FORWARD);

  Serial.printf("[SERVO] Triggering dispense for Bowl %d. Current: %.1fg, Target: %.1fg\n",
bowlNumber, currentWeight, disp->targetTotalWeight);
}
```

```
void handleDispensers() {
  DispenserState* dispensers[] = {&dispenser1, &dispenser2};
  HX711* scales[] = {&loadCellBowl1, &loadCellBowl2};

  for (int i = 0; i < 2; i++) {
    DispenserState* disp = dispensers[i];
    HX711* scale = scales[i];
    unsigned long currentMillis = millis();

    if (disp->mode == D_DISPENSING) {
      float currentWeight = getWeightInGrams(*scale, disp->bowlId);

      Serial.printf("[TEST] Dispensing Bowl %d | Current: %.1fg | Target: %.1fg\n", disp->bowlId,
currentWeight, disp->targetTotalWeight);

      if (currentWeight >= disp->targetTotalWeight) {
        disp->servo->write(SERVO_STOP);
        disp->mode = D_IDLE;
        Serial.printf("[SERVO] Bowl %d Target Reached! (%.1fg). Dispense complete.\n", disp-
>bowlId, currentWeight);
        updateFoodLevelsInFirestore();
        return;
      }

      if (currentWeight > disp->lastStableWeight + MIN_WEIGHT_CHANGE_THRESHOLD) {
        disp->lastStableWeight = currentWeight;
        disp->lastWeightChangeTime = currentMillis;
      }
      else if (currentMillis - disp->lastWeightChangeTime > STALL_TIMEOUT_MS) {
```

```
        if (disp->unclogAttempts < MAX_UNCLOG_ATTEMPTS) {
          Serial.printf("[SERVO] Bowl %d JAM DETECTED (No weight increase for 30s). Starting UNCLOG routine (Attempt %d/%d).\n", disp->bowlId, disp->unclogAttempts + 1, MAX_UNCLOG_ATTEMPTS);

          disp->mode = D_UNCLOGGING;
          disp->unclogStep = U_STEP_1_REVERSE;
          disp->stateStartTime = currentMillis;
          disp->servo->write(SERVO_REVERSE);

          disp->unclogAttempts++;
        } else {
          Serial.printf("[SERVO] Bowl %d FAILED. Container likely empty or jamming persists after 3 tries. Stopping.\n", disp->bowlId);
          disp->servo->write(SERVO_STOP);
          disp->mode = D_IDLE;
          sendEmptyErrorNotification(disp->bowlId);
        }
      }
    }
    else if (disp->mode == D_UNCLOGGING) {
      unsigned long stepDuration = 2500;

      if (currentMillis - disp->stateStartTime > stepDuration) {
        disp->stateStartTime = currentMillis;

        switch (disp->unclogStep) {
          case U_STEP_1_REVERSE:
            Serial.println("[UNCLOG] Step 1 complete. Switching to FORWARD.");
            disp->servo->write(SERVO_FORWARD);
```

```
          disp->unclogStep = U_STEP_2_FORWARD;
          break;
        case U_STEP_2_FORWARD:
          Serial.println("[UNCLOG] Step 2 complete. Switching to REVERSE.");
          disp->servo->write(SERVO_REVERSE);
          disp->unclogStep = U_STEP_3_REVERSE;
          break;
        case U_STEP_3_REVERSE:
          Serial.println("[UNCLOG] Step 3 complete. Switching to FORWARD.");
          disp->servo->write(SERVO_FORWARD);
          disp->unclogStep = U_STEP_4_FORWARD;
          break;
        case U_STEP_4_FORWARD:
          Serial.println("[UNCLOG] Sequence Complete. Returning to normal Dispensing.");
          disp->mode = D_DISPENSING;
          disp->lastWeightChangeTime = currentMillis;
          disp->lastStableWeight = getWeightInGrams(*scale, disp->bowlId);
          break;
      }
    }
  }
}

void sendEmptyErrorNotification(int bowlId) {
  if (feederId.length() == 0) return;
  String path = "feeder_errors/";
  path += feederId;
  path += "/";
```

```
  path += millis();

  FirebaseJson json;
  json.set("bowl", bowlId);
  json.set("error", "CONTAINER_EMPTY_OR_JAMMED");
  json.set("timestamp", (unsigned long)millis());

  fbdo.clear();
  if (Firebase.RTDB.setJSON(&fbdo, path.c_str(), &json)) {
    Serial.println("[ERROR] Notification sent to RTDB.");
  }
}

void sendHungerAlert(int bowlId, String tagId) {
  if (feederId.length() == 0) return;
  String path = "feeder_hunger_alerts/";
  path += feederId;
  path += "/";
  path += millis();

  FirebaseJson json;
  json.set("bowl", bowlId);
  json.set("tagId", tagId);
  json.set("timestamp", (unsigned long)millis());

  fbdo.clear();
  if (Firebase.RTDB.setJSON(&fbdo, path.c_str(), &json)) {
    Serial.println("[ALERT] Hunger alert sent to RTDB.");
  }
```

```
}

void updateBowlStatus(int bowl, bool isWaiting, int amount, String petId) {
  if (feederId.length() == 0) return;
  String path = "/feeders/";
  path += feederId;
  path += "/bowlStatus/";
  path += bowl;

  FirebaseJson json;
  json.set("isWaiting", isWaiting);

  if (isWaiting) {
    json.set("amount", amount);
    json.set("petId", petId);
  } else {
    json.set("amount", 0);
    json.set("petId", "");
  }

  fbdo.clear();
  if (Firebase.RTDB.setJSON(&fbdo, path.c_str(), &json)) {
  }
}

void startRTDBStream() {
  if (feederId.length() == 0 || !Firebase.ready()) return;

  String statusPath = "/feeders/";
```

```
  statusPath += feederId;
  statusPath += "/status";

  Serial.println("[PRESENCE] Setting status to online...");
  fbdo.clear();

  if (Firebase.RTDB.setString(&fbdo, statusPath.c_str(), "online")) {
    Serial.println("[PRESENCE] Status set to online.");
  } else {
    Serial.printf("[PRESENCE] Failed to set online status: %s\n", fbdo.errorReason().c_str());
  }

  String fullStreamPath = streamPath;
  fullStreamPath.concat(feederId);

  Serial.print("[STREAM] Starting stream on path: ");
  Serial.println(fullStreamPath);

  stream_fbdo.clear();
  if (!Firebase.RTDB.beginStream(&stream_fbdo, fullStreamPath.c_str())) {
    Serial.printf("[STREAM] Could not begin stream: %s\n", stream_fbdo.errorReason().c_str());
    return;
  }

  Firebase.RTDB.setStreamCallback(&stream_fbdo, streamCallback, streamTimeoutCallback);
  streamActive = true;
  Serial.println("[STREAM] Stream started successfully.");
}
```

```
void streamCallback(FirebaseStream data) {
  Serial.printf("[STREAM] Event: %s, Path: %s, Data: %s\n", data.eventType().c_str(), data.dataPath().c_str(), data.payload().c_str());

  if (data.dataType() == "json" && data.dataPath() == "/") {
    FirebaseJson json;
    json.setJsonData(data.payload());
    FirebaseJsonData result;

    String command;
    if (json.get(result, "command")) {
      command = result.to<String>();
    }

    if (command == "feed") {
      unsigned long newTimestamp = 0;
      int bowl = 0;
      int amount = 0;

      if (json.get(result, "timestamp")) newTimestamp = result.to<unsigned long>();
      if (json.get(result, "bowl")) bowl = result.to<int>();
      if (json.get(result, "amount")) amount = result.to<int>();

      if (newTimestamp > 0 && newTimestamp >= lastCmdTimestamp) {
        lastCmdTimestamp = newTimestamp;
        pendingCommand = {bowl, amount};
        newCommandAvailable = true;

        if (bowl == 1 && bowl1State.active) {
```

```
      Serial.println("[RFID] Feed Now for Bowl 1 received, cancelling pending schedule.");
      bowl1State.active = false;
      statusUpdateQueue.bowl = 1;
      statusUpdateQueue.isWaiting = false;
      statusUpdateQueue.amount = 0;
      statusUpdateQueue.petId = "";
      statusUpdateAvailable = true;
    }
    if (bowl == 2 && bowl2State.active) {
      Serial.println("[RFID] Feed Now for Bowl 2 received, cancelling pending schedule.");
      bowl2State.active = false;
      statusUpdateQueue.bowl = 2;
      statusUpdateQueue.isWaiting = false;
      statusUpdateQueue.amount = 0;
      statusUpdateQueue.petId = "";
      statusUpdateAvailable = true;
    }
  }
}
 else if (command == "scan_tag_bowl_1") {
  Serial.println("[RFID] Entering PAIRING mode for BOWL 1 by app request.");
  globalMode = PAIRING;
  pairingBowlTarget = 1;
  pairingStartTime = millis();
  clearCommandNode = true;
}
 else if (command == "scan_tag_bowl_2") {
  Serial.println("[RFID] Entering PAIRING mode for BOWL 2 by app request.");
  globalMode = PAIRING;
```

```
      pairingBowlTarget = 2;
      pairingStartTime = millis();
      clearCommandNode = true;
    }
    else if (command == "cancel_scan_bowl_1" || command == "cancel_scan_bowl_2") {
      if (globalMode == PAIRING) {
        Serial.printf("[RFID] Cancelling PAIRING mode for bowl %d.\n", (command ==
"cancel_scan_bowl_1" ? 1 : 2));
        globalMode = NORMAL;
        pairingBowlTarget = 0;
      }
      clearCommandNode = true;
    }
    else if (command == "await_rfid") {
      unsigned long newTimestamp = 0;
      if (json.get(result, "timestamp")) newTimestamp = result.to<unsigned long>();

      if (newTimestamp > 0 && newTimestamp >= lastCmdTimestamp) {
        lastCmdTimestamp = newTimestamp;
        int bowl = 0;
        int amount = 0;
        String tag = "";
        String scheduleId = "";
        String petId = "";

        if (json.get(result, "bowl")) bowl = result.to<int>();
        if (json.get(result, "amount")) amount = result.to<int>();
        if (json.get(result, "expectedTagId")) tag = result.to<String>();
        if (json.get(result, "scheduleId")) scheduleId = result.to<String>();
```

```
    if (json.get(result, "petId")) petId = result.to<String>();


    if (bowl > 0 && amount > 0 && tag.length() > 0 && scheduleId.length() > 0 && petId.length()
> 0) {
      Serial.printf("[RFID] Entering WAITING mode for Bowl %d, Tag: %s, Schedule: %s\n",
bowl, tag.c_str(), scheduleId.c_str());


      if (bowl == 1) {
        bowl1State.active = true;
        bowl1State.expectedTag = tag;
        bowl1State.amount = amount;
        bowl1State.scheduleId = scheduleId;
        bowl1State.petId = petId;
        bowl1State.waitStartTime = millis();
        Serial.println("[RFID] Bowl 1 is now active.");


        statusUpdateQueue.bowl = 1;
        statusUpdateQueue.isWaiting = true;
        statusUpdateQueue.amount = amount;
        statusUpdateQueue.petId = petId;
        statusUpdateAvailable = true;
      } else if (bowl == 2) {
        bowl2State.active = true;
        bowl2State.expectedTag = tag;
        bowl2State.amount = amount;
        bowl2State.scheduleId = scheduleId;
        bowl2State.petId = petId;
        bowl2State.waitStartTime = millis();
        Serial.println("[RFID] Bowl 2 is now active.");
```

```
          statusUpdateQueue.bowl = 2;
          statusUpdateQueue.isWaiting = true;
          statusUpdateQueue.amount = amount;
          statusUpdateQueue.petId = petId;
          statusUpdateAvailable = true;
        }

        clearCommandNode = true;
      } else {
        Serial.println("[RFID] Invalid await_rfid command received (missing parameters).");
        clearCommandNode = true;
      }
    }
  }
  else if (command == "reset_device") {
    Serial.println("[SYSTEM] Reset command received. Transitioning to RESETTING state.");
    currentState = RESETTING;
  }
  else if (command.length() > 0) {
    Serial.printf("[SYSTEM] Unknown command received: %s. Deleting.\n", command.c_str());
    clearCommandNode = true;
  }
}
else if (data.eventType() == "put" && data.dataPath() == "/" && data.stringData() == "null") {
  if (globalMode == PAIRING) {
    Serial.println("[RFID] Command node cleared. Continuing PAIRING mode...");
  }
}
```

```
}

void streamTimeoutCallback(bool timeout) {
  if (timeout) {
    Serial.println("[STREAM] Stream timed out. It will be restarted automatically.");
    streamActive = false;
  }
}

void forwardSlaveLogs() {
  if (Serial1.available()) {
    Serial.write(Serial1.read());
  }
  if (Serial2.available()) {
    Serial.write(Serial2.read());
  }
}

bool sendToSlave(HardwareSerial& serial, const String& packet) {
  for (int i = 0; i < 5; i++) {
    Serial.printf("  Attempt %d to send...\n", i + 1);
    serial.print(packet);
    serial.flush();

    unsigned long start = millis();
    while (millis() - start < 1000) {
      if (serial.available()) {
        String response = serial.readStringUntil('\n');
        response.trim();
```

```
      if (response == "OK") {
        return true;
      }
    }
  }
  delay(200);
 }
 return false;
}

float getWeightInGrams(HX711& sensor, int sensorNum) {
 if (sensor.is_ready()) {
  float weightGrams = sensor.get_units(1);
  if (weightGrams < 0) weightGrams = 0;
  return weightGrams;
 } else {
  return 0;
 }
}

void updateFoodLevelsInFirestore() {
 if (!Firebase.ready() || feederId.length() == 0) {
  Serial.println("[FIREBASE] Not ready to update food levels.");
  return;
 }

 float weight1 = getWeightInGrams(loadCellBowl1, 1);
 float weight2 = getWeightInGrams(loadCellBowl2, 2);
```

```
  Serial.printf("[FIREBASE] Updating food levels (BOWL WEIGHT). Bowl 1: %.0fg, Bowl 2: %.0fg\n", weight1, weight2);

  String feederPath = "feeders/";
  feederPath.concat(feederId);

  FirebaseJson content;
  int weight1_int = (int)weight1;
  int weight2_int = (int)weight2;

  content.set("fields/foodLevels/mapValue/fields/1/integerValue", weight1_int);
  content.set("fields/foodLevels/mapValue/fields/2/integerValue", weight2_int);

  String contentStr;
  content.toString(contentStr, false);

  fbdo.clear();
  if (Firebase.Firestore.patchDocument(&fbdo, FIREBASE_PROJECT_ID, "", feederPath.c_str(), contentStr.c_str(), "foodLevels")) {
    Serial.println("[FIREBASE] Food levels updated successfully.");
  } else {
    Serial.printf("[ERROR] Failed to update food levels: %s\n", fbdo.errorReason().c_str());
  }
}

// --- RFID READING LOGIC ---

void updateRfidBuffers() {
  while (SerialRFID1.available() > 0) {
```

```
  char c = SerialRFID1.read();
  if (rfid1_buffer.length() < 50) {
    rfid1_buffer += c;
  }
  lastRfid1ByteTime = millis();
}

while (SerialRFID2.available() > 0) {
  char c = SerialRFID2.read();
  if (rfid2_buffer.length() < 50) {
    rfid2_buffer += c;
  }
  lastRfid2ByteTime = millis();
}

if (rfid1_buffer.length() > 0 && (millis() - lastRfid1ByteTime > rfidTagTimeout)) {
  rfid1_buffer.trim();
  if (rfid1_buffer.length() > 0) {
    handleRfidRead(1, rfid1_buffer);
  }
  rfid1_buffer = "";
}

if (rfid2_buffer.length() > 0 && (millis() - lastRfid2ByteTime > rfidTagTimeout)) {
  rfid2_buffer.trim();
  if (rfid2_buffer.length() > 0) {
    handleRfidRead(2, rfid2_buffer);
  }
  rfid2_buffer = "";
```

```
  }
}

void handleRfidRead(int readerNum, String scannedTagId) {
  String cleanedTagId = "";

  for (size_t i = 0; i < scannedTagId.length(); i++) {
    char c = scannedTagId.charAt(i);
    if ((c >= 'A' && c <= 'F') || (c >= 'a' && c <= 'f') || (c >= '0' && c <= '9')) {
      cleanedTagId += c;
    }
  }

  if (cleanedTagId.length() < 8 || cleanedTagId.length() > 32) {
    Serial.printf("[RFID] Ignored noise (Length %d): %s\n", cleanedTagId.length(),
cleanedTagId.c_str());
    return;
  }

  if (cleanedTagId == lastScannedTag && (millis() - lastLogTime < 2000)) {
    lastLogTime = millis();
    return;
  }

  lastScannedTag = cleanedTagId;
  lastLogTime = millis();

  Serial.printf("[RFID] Reader %d Scanned! Raw: %s, Cleaned: %s\n", readerNum,
scannedTagId.c_str(), cleanedTagId.c_str());
```

```
  if (globalMode == PAIRING) {
    if (pairingBowlTarget == readerNum) {
      Serial.printf("[RFID] Tag captured for PAIRING Bowl %d. Sending to app...\n", readerNum);
      String scanPath = "scan_pairing/";
      scanPath.concat(feederId);
      scanPath.concat("/bowl_");
      scanPath.concat(readerNum);

      FirebaseJson content;
      content.set("tagId", cleanedTagId);
      content.set("timestamp", (unsigned long)millis());

      bool success = false;

      for (int i = 0; i < 10; i++) {
        fbdo.clear();
        if (Firebase.RTDB.setJSON(&fbdo, scanPath.c_str(), &content)) {
          success = true;
          break;
        } else {
          Serial.printf("[RFID] Retrying send... (%d/10) - Reason: %s\n", i + 1,
fbdo.errorReason().c_str());
          delay(1000);
        }
      }

      if (success) {
        globalMode = NORMAL;
```

```
      pairingBowlTarget = 0;
    } else {
      Serial.printf("[ERROR] Failed to send pairing data: %s\n", fbdo.errorReason().c_str());
    }
  }
  return;
 }

 if (readerNum == 1 && bowl1State.active) {
  Serial.printf("[RFID] Checking Bowl 1 Schedule. Expected: %s, Scanned: %s\n",
bowl1State.expectedTag.c_str(), cleanedTagId.c_str());
  if (cleanedTagId.equalsIgnoreCase(bowl1State.expectedTag)) {
    Serial.println("[RFID] Bowl 1 MATCH! Triggering dispense.");
    triggerDispense(1, bowl1State.amount);
    bowl1State.active = false;
    unscheduledScansBowl1 = 0;

    statusUpdateQueue.bowl = 1;
    statusUpdateQueue.isWaiting = false;
    statusUpdateQueue.amount = 0;
    statusUpdateQueue.petId = "";
    statusUpdateAvailable = true;
    clearCommandNode = true;
  }
 } else if (readerNum == 2 && bowl2State.active) {
  Serial.printf("[RFID] Checking Bowl 2 Schedule. Expected: %s, Scanned: %s\n",
bowl2State.expectedTag.c_str(), cleanedTagId.c_str());
  if (cleanedTagId.equalsIgnoreCase(bowl2State.expectedTag)) {
    Serial.println("[RFID] Bowl 2 MATCH! Triggering dispense.");
```

```
    triggerDispense(2, bowl2State.amount);
    bowl2State.active = false;
    unscheduledScansBowl2 = 0;

    statusUpdateQueue.bowl = 2;
    statusUpdateQueue.isWaiting = false;
    statusUpdateQueue.amount = 0;
    statusUpdateQueue.petId = "";
    statusUpdateAvailable = true;
    clearCommandNode = true;
  }
} else {
  Serial.println("[RFID] Tag scanned but no active schedule for this bowl.");

  if (readerNum == 1) {
    unscheduledScansBowl1++;
    Serial.printf("[RFID] Bowl 1 unscheduled scan count: %d\n", unscheduledScansBowl1);
    if (unscheduledScansBowl1 >= 3) {
      Serial.println("[RFID] Hunger threshold reached for Bowl 1! Sending alert.");
      sendHungerAlert(1, cleanedTagId);
      unscheduledScansBowl1 = 0;
    }
  } else if (readerNum == 2) {
    unscheduledScansBowl2++;
    Serial.printf("[RFID] Bowl 2 unscheduled scan count: %d\n", unscheduledScansBowl2);
    if (unscheduledScansBowl2 >= 3) {
      Serial.println("[RFID] Hunger threshold reached for Bowl 2! Sending alert.");
      sendHungerAlert(2, cleanedTagId);
      unscheduledScansBowl2 = 0;
```

```
    }
   }
  }
}

void checkRfidTimeouts() {
  unsigned long currentMillis = millis();

  if (bowl1State.active && (currentMillis - bowl1State.waitStartTime > 120000)) {
   Serial.printf("[RFID] WAITING mode for Bowl 1 (Schedule %s) timed out.\n",
bowl1State.scheduleId.c_str());
   String timeoutPath = "feeder_timeout_events/";
   timeoutPath.concat(feederId);
   timeoutPath.concat("/");
   timeoutPath.concat(String(currentMillis));

   FirebaseJson content;
   content.set("scheduleId", bowl1State.scheduleId);
   content.set("petId", bowl1State.petId);
   content.set("timestamp", (unsigned long)currentMillis);

   fbdo.clear();
   if (Firebase.RTDB.setJSON(&fbdo, timeoutPath.c_str(), &content)) {
    Serial.println("[RFID] Timeout 1 reported to RTDB.");
   }

   bowl1State.active = false;
   statusUpdateQueue.bowl = 1;
   statusUpdateQueue.isWaiting = false;
```

```
    statusUpdateQueue.amount = 0;
    statusUpdateQueue.petId = "";
    statusUpdateAvailable = true;
  }

  if (bowl2State.active && (currentMillis - bowl2State.waitStartTime > 120000)) {
    Serial.printf("[RFID] WAITING mode for Bowl 2 (Schedule %s) timed out.\n",
bowl2State.scheduleId.c_str());
    String timeoutPath = "feeder_timeout_events/";
    timeoutPath.concat(feederId);
    timeoutPath.concat("/");
    timeoutPath.concat(String(currentMillis));

    FirebaseJson content;
    content.set("scheduleId", bowl2State.scheduleId);
    content.set("petId", bowl2State.petId);
    content.set("timestamp", (unsigned long)currentMillis);

    fbdo.clear();
    if (Firebase.RTDB.setJSON(&fbdo, timeoutPath.c_str(), &content)) {
      Serial.println("[RFID] Timeout 2 reported to RTDB.");
    }

    bowl2State.active = false;
    statusUpdateQueue.bowl = 2;
    statusUpdateQueue.isWaiting = false;
    statusUpdateQueue.amount = 0;
    statusUpdateQueue.petId = "";
    statusUpdateAvailable = true;
```

```
  }

  if (globalMode == PAIRING && (currentMillis - pairingStartTime > 30000)) {
    Serial.printf("[RFID] PAIRING mode for bowl %d timed out.\n", pairingBowlTarget);
    globalMode = NORMAL;
    pairingBowlTarget = 0;
  }
}
```

Slave file

```
#include "esp_camera.h"
#include "Arduino.h"
#include "FS.h"
#include "soc/soc.h"
#include "soc/rtc_cntl_reg.h"
#include "driver/rtc_io.h"
#include <WiFi.h>
#include <Preferences.h>
#include "esp_http_server.h"
#include <Firebase_ESP_Client.h>
#include <time.h>
#include <ESPmDNS.h>
#include "lwip/dns.h"
#include "lwip/ip_addr.h"
#include "addons/RTDBHelper.h"

#define BOWL_NUMBER 1

#define FIREBASE_PROJECT_ID "pawfeeds-v2"
```

#define FIREBASE_DATABASE_URL "https://pawfeeds-v2-default-rtdb.asia-southeast1.firebasedatabase.app/"

#define SERVICE_ACCOUNT_CLIENT_EMAIL "firebase-adminsdk-fbsvc@pawfeeds-v2.iam.gserviceaccount.com"

#define SERVICE_ACCOUNT_PRIVATE_KEY "-----BEGIN PRIVATE KEY-----\n" \

"MIIEvwIBADANBgkqhkiG9w0BAQEFAASCBKkwggSlAgEAAoIBAQC+Sqt8BKfCEgbp\n" \

"/q4XmzzpsKwlsKJesbWN/Sp2PO7nVgPqwEz9YeCy/aI98IjyGY5hujaLLNgB0GYE\n" \

"NLvWkAf3T6VQPUa8SB28XU5sT7tfCJu/1F3yYYAVHHpbg2gbGxjgPWVlQ5arww57\n" \

"HHREzOUTRTO5k9hRAzzf1kdjBJbboewEEcIQH2uAdn+B9L58SOezjVONpr63+R5V\n" \

"WbZ7+B64BFE62Xsjcmk49OmHjKc+2ID80S6EWx6rxM5jGdTsUVtA0RISG5r1qeB/\n" \

"DazLCx1jexutuMHTogvAIirHGIMdOCW12USD8b+CRvP7Lsj6CwR91JSVm/b5P1pI\n" \

"0MNAgkfrAgMBAAECggEAO5iNHIki5P/aVHxjr5b5u8KOF3u7TmbfkmmAW+l3dNIW\n" \

"hfXV5uE5izUuE7H6YcApPGgiXvIbcG4BFT4iue7/3698+aVHOv5m+bBLOFa8OuYq\n" \

"SSjMh3WLtJDnrTN5bkvNPaVc1RsW3BJJvbrKmyWEdMWOjodEDxMxhHTKhLNSP9Rq\n" \

"cTz85kc4/IsXuAD6fmApysrWiVuS+vclzFGCjqQDozojsd58KW6XFa8PEZ95bUbE\n" \

"/Q8ZAigB88un8+87NpLd/usUk2FWHNJz523AJUA4qQE0N8J6E/7+dJbRJ9Ucg6lk\n" \

"kO0fRiLO7c7o6/7bW1iy4Yq+ph5N3+vc2pYzVZlkAQKBgQD0AbH86UoJxroGh/id\n" \

"849814dnEGugSQ1C2XCPNpQy19WtyfZvMM5yphfxfxxohJwr+KyQBcfBatGo/ipV\n" \

"9clJUOalKa+q9KzxLsSka41cvu4RsnVjWyVqGS4w/BdYFdTvNkuK3cGiK8qYwBaW\n" \

"i50PNrcCQ+7gmMCjkPJ6LDOh6wKBgQDHpRYRXk9ipsOw4KHBxn4rph8G6ooUIvzs\n" \

"Pie0e+A+pz9grNDC7y0a+k8fA4eq/iJPmu+FvuJW4fahCkt98VuW0EJjcqtBNgCB\n" \

"YbOKn9H+V4WaQKCHYyhl/QxGio4qm4z34cEVt/kMGwGHlaX39RILc3kEq2RfTSc7\n" \

"2FikgZxyAQKBgQCz61wWpN5W/xXEIxaLQUCYSUQqFs2FTthcZoC82P3Fz6hbkQQJ\n" \

"UO+pUhdtltCXsNCHC8ISIHD+iYk3FtKYt7HvtJudRXOmluu+m0GcC0IdFRvuKKyu\n" \

"KlMYPKD2tatw5AgyqtJg/sr8jVXB9EGzmBajVTD0lqrZKUlCUmq480bPKQKBgQCp\n" \

"To0qaYp1JOur4rQK+uQg7B4/5UL31LwdNJDDdJI1T+xldekMh3z+9euHZ5z0G9TJ\n" \

"IaGjEMAt4i8fXvWqdravbSn/4EzvXnaLQmnaU7LoORzqNYhtiF/ILhLs96+c3pFr\n" \

"3h2652vjljvn2bcIUuLcpy6oERlr4Kg3DkAOMoSUAQKBgQCkTg6e4Ndg1d2SySFQ\n" \

```
"4h7YHQZ3larxK/03DFoElNIf/6zgiNL13+9nM3l4K8YAEG1OEcn5JaBdktmGrEWR\n" \
"A+zV7Qp/hZSVuOM4sxpGa7q94A5253tvLvegIvzhFYW4WQrVFadI5xZ1HaO2Ad9I\n" \
"Kzz19+jbd085r+kyTM5X0UVzZA==\n" \
"-----END PRIVATE KEY-----\n"

#define PWDN_GPIO_NUM    32
#define RESET_GPIO_NUM   -1
#define XCLK_GPIO_NUM     0
#define SIOD_GPIO_NUM    26
#define SIOC_GPIO_NUM    27
#define Y9_GPIO_NUM      35
#define Y8_GPIO_NUM      34
#define Y7_GPIO_NUM      39
#define Y6_GPIO_NUM      36
#define Y5_GPIO_NUM      21
#define Y4_GPIO_NUM      19
#define Y3_GPIO_NUM      18
#define Y2_GPIO_NUM      5
#define VSYNC_GPIO_NUM   25
#define HREF_GPIO_NUM    23
#define PCLK_GPIO_NUM    22

Preferences preferences;
FirebaseData stream_fbdo;
FirebaseData upload_fbdo;
FirebaseAuth auth;
FirebaseConfig config;
String feederId = "";
bool streamActive = false;
```

```
bool cameraTaskStarted = false;
bool ipUploaded = false;

enum DeviceState { PROVISIONING, CONNECTING_WIFI, AWAITING_FEEDER_ID,
SYNCING_TIME, AUTHENTICATING_FIREBASE, OPERATIONAL, ERROR };
DeviceState currentState = PROVISIONING;

#if BOWL_NUMBER == 1
  HardwareSerial SerialSlave(2);
  const int RX_PIN = 12;
  const int TX_PIN = 13;
#elif BOWL_NUMBER == 2
  HardwareSerial SerialSlave(1);
  const int RX_PIN = 14;
  const int TX_PIN = 15;
#else
  #error "BOWL_NUMBER must be 1 or 2"
#endif

// --- Prototypes ---
void listenForCredentials();
void connectToWiFi();
void syncTime();
void startCameraServer();
void authenticateWithFirebase();
void startRTDBStream();
void streamCallback(FirebaseStream data);
void streamTimeoutCallback(bool timeout);
void tokenStatusCallback(TokenInfo info);
```

```
void listenForFeederId();
void uploadCurrentIP();
esp_err_t stream_handler(httpd_req_t *req);
void camera_server_task(void *pvParameters);
void performFactoryReset();
void checkSerialCommands();

void setup() {
  WRITE_PERI_REG(RTC_CNTL_BROWN_OUT_REG, 0);
  Serial.begin(115200);
  Serial.println("\n[PawFeeds CAM] Booting (High FPS Mode)...");
  SerialSlave.begin(9600, SERIAL_8N1, RX_PIN, TX_PIN);

  preferences.begin("pawfeeds-cam", true);
  String ssid = preferences.getString("ssid", "");
  feederId = preferences.getString("feederId", "");
  preferences.end();

  if (ssid.length() == 0) {
    currentState = PROVISIONING;
  } else {
    currentState = CONNECTING_WIFI;
  }
}

void performFactoryReset() {
  Serial.println("[SYSTEM] Factory Reset triggered...");
  preferences.begin("pawfeeds-cam", false);
  preferences.clear();
```

```
  preferences.end();
  delay(1000);
  ESP.restart();
}

void checkSerialCommands() {
  if (SerialSlave.available()) {
    String packet = SerialSlave.readStringUntil('\n');
    packet.trim();
    if (packet.length() > 0) {
      Serial.printf("[SERIAL] Received: %s\n", packet.c_str());
      if (packet == "RESET_DEVICE") {
        SerialSlave.println("OK");
        delay(100);
        performFactoryReset();
      }
    }
  }
}

// --- MAIN LOOP ---
void loop() {
  switch(currentState) {
    case PROVISIONING:
      listenForCredentials();
      break;
    case CONNECTING_WIFI:
      connectToWiFi();
      break;
```

```
    case AWAITING_FEEDER_ID:
      listenForFeederId();
      break;
    case SYNCING_TIME:
      syncTime();
      break;
    case AUTHENTICATING_FIREBASE:
      authenticateWithFirebase();
      break;
    case OPERATIONAL:
      if (!cameraTaskStarted) {
        Serial.println("[SYSTEM] Starting camera task on Core 1.");
        xTaskCreatePinnedToCore(camera_server_task, "CameraServer", 10000, NULL, 2, NULL,
1);
        cameraTaskStarted = true;
      }
      if (!streamActive) {
         startRTDBStream();
      }
      if (!ipUploaded && Firebase.ready() && feederId.length() > 0) {
        uploadCurrentIP();
      }
      checkSerialCommands();
      break;
    case ERROR:
      delay(5000);
      ESP.restart();
      break;
  }
```

```
  delay(100);
}

void uploadCurrentIP() {
    String ipAddress = WiFi.localIP().toString();

    String nodePath = "/feeders/";
    nodePath += feederId;
    nodePath += "/camIp";
    nodePath += (BOWL_NUMBER == 1 ? "1" : "2");

    Serial.printf("[IP SYNC] Uploading IP %s to %s\n", ipAddress.c_str(), nodePath.c_str());

    if (Firebase.RTDB.setString(&upload_fbdo, nodePath, ipAddress)) {
        Serial.println("[IP SYNC] IP Address uploaded successfully.");
        ipUploaded = true;
    } else {
        Serial.printf("[IP SYNC] Failed to upload IP: %s\n", upload_fbdo.errorReason().c_str());
        ipUploaded = false;
    }
}

void listenForCredentials() {
  if (SerialSlave.available() > 0) {
      String packet = SerialSlave.readStringUntil('\n');
      if (packet.indexOf("FEEDER_ID") != -1) return;

      int separator = packet.indexOf('|');
      if (separator > 0) {
```

```
      String ssid = packet.substring(0, separator);
      String pass = packet.substring(separator + 1);
      pass.trim();

      Serial.println("[PROVISIONING] Credentials received. Saving...");
      preferences.begin("pawfeeds-cam", false);
      preferences.putString("ssid", ssid);
      preferences.putString("pass", pass);
      preferences.end();
      SerialSlave.println("OK");
      delay(1000);
      ESP.restart();
    }
  }
}

void listenForFeederId() {
   if (SerialSlave.available() > 0) {
     String packet = SerialSlave.readStringUntil('\n');
     if (packet.startsWith("FEEDER_ID|")) {
       String receivedId = packet.substring(packet.indexOf('|') + 1);
       receivedId.trim();
       preferences.begin("pawfeeds-cam", false);
       preferences.putString("feederId", receivedId);
       preferences.end();
       feederId = receivedId;
       SerialSlave.println("OK");
       currentState = SYNCING_TIME;
     }
```

```
    }
}

void connectToWiFi() {
  preferences.begin("pawfeeds-cam", true);
  String ssid = preferences.getString("ssid");
  String pass = preferences.getString("pass");
  preferences.end();

  WiFi.mode(WIFI_STA);
  WiFi.begin(ssid.c_str(), pass.c_str());

  int attempts = 0;
  while (WiFi.status() != WL_CONNECTED && attempts < 20) {
    delay(500);
    Serial.print(".");
    attempts++;
  }

  if (WiFi.status() == WL_CONNECTED) {
    Serial.print("\n[Wi-Fi] Connected! IP: ");
    Serial.println(WiFi.localIP());

    String hostname = "pawfeeds-cam-";
    hostname += BOWL_NUMBER;

    if (MDNS.begin(hostname.c_str())) {
        Serial.printf("[MDNS] Started: %s.local\n", hostname.c_str());
        MDNS.addService("http", "tcp", 80);
```

```
    }

    if (feederId.length() == 0) {
      currentState = AWAITING_FEEDER_ID;
    } else {
      currentState = SYNCING_TIME;
    }
  } else {
    Serial.println("\n[Wi-Fi] Failed. Rebooting.");
    ESP.restart();
  }
}

void syncTime() {
    Serial.println("[TIME] Syncing...");
    configTime(8 * 3600, 0, "pool.ntp.org", "time.nist.gov");
    time_t now = time(nullptr);
    int attempts = 0;
    while (now < 8 * 3600 * 2 && attempts < 20) {
        delay(500);
        now = time(nullptr);
        attempts++;
    }
    currentState = AUTHENTICATING_FIREBASE;
}

void authenticateWithFirebase() {
 if(Firebase.ready()) {
    currentState = OPERATIONAL;
```

```
    return;
  }
  Serial.println("[AUTH] Auth init...");
  config.database_url = FIREBASE_DATABASE_URL;
  config.service_account.data.client_email = SERVICE_ACCOUNT_CLIENT_EMAIL;
  config.service_account.data.project_id = FIREBASE_PROJECT_ID;
  config.service_account.data.private_key = SERVICE_ACCOUNT_PRIVATE_KEY;
  config.token_status_callback = tokenStatusCallback;

  Firebase.begin(&config, &auth);
  Firebase.reconnectWiFi(true);
}

void tokenStatusCallback(TokenInfo info) {
  if (info.status == token_status_ready) {
    Serial.println("[AUTH] Token ready.");
    currentState = OPERATIONAL;
  }
}

void startRTDBStream() {
    if (feederId.length() == 0 || !Firebase.ready()) return;

    String fullStreamPath = "/commands/";
    fullStreamPath += feederId;

    if (Firebase.RTDB.beginStream(&stream_fbdo, fullStreamPath.c_str())) {
        Firebase.RTDB.setStreamCallback(&stream_fbdo, streamCallback,
streamTimeoutCallback);
```

```
        streamActive = true;
    }
}

void streamCallback(FirebaseStream data) {
    if (data.dataType() == "json") {
        FirebaseJson &json = data.to<FirebaseJson>();
        FirebaseJsonData result;
        if (json.get(result, "command")) {
            if (result.to<String>() == "reset_device") {
                String cmdPath = "/commands/";
                cmdPath += feederId;

                Firebase.RTDB.deleteNode(&stream_fbdo, cmdPath);
                performFactoryReset();
            }
        }
    }
}

void streamTimeoutCallback(bool timeout) {
  if (timeout) streamActive = false;
}

// --- Camera Logic ---
#define PART_BOUNDARY "123456789000000000000000000000000"
static const char* _STREAM_CONTENT_TYPE = "multipart/x-mixed-replace;boundary="
PART_BOUNDARY;
static const char* _STREAM_BOUNDARY = "\r\n--" PART_BOUNDARY "\r\n";
```

```
static const char* _STREAM_PART = "Content-Type: image/jpeg\r\nContent-Length: %u\r\n\r\n";

esp_err_t stream_handler(httpd_req_t *req){
  camera_fb_t * fb = NULL;
  esp_err_t res = ESP_OK;
  size_t _jpg_buf_len = 0;
  uint8_t * _jpg_buf = NULL;
  char * part_buf[64];

  res = httpd_resp_set_type(req, _STREAM_CONTENT_TYPE);
  if (res != ESP_OK) return res;

  while (true) {
    fb = esp_camera_fb_get();
    if (!fb) {
      res = ESP_FAIL;
      break;
    }

    _jpg_buf_len = fb->len;
    _jpg_buf = fb->buf;

    if (res == ESP_OK) res = httpd_resp_send_chunk(req, _STREAM_BOUNDARY,
strlen(_STREAM_BOUNDARY));
    if (res == ESP_OK) {
      size_t hlen = snprintf((char *)part_buf, 64, _STREAM_PART, _jpg_buf_len);
      res = httpd_resp_send_chunk(req, (const char *)part_buf, hlen);
    }
    if (res == ESP_OK) res = httpd_resp_send_chunk(req, (const char *)_jpg_buf, _jpg_buf_len);
```

```
    esp_camera_fb_return(fb);
    if (res != ESP_OK) break;

    delay(1);
  }
  return res;
}

void startCameraServer(){
  camera_config_t config;
  config.ledc_channel = LEDC_CHANNEL_0;
  config.ledc_timer = LEDC_TIMER_0;
  config.pin_d0 = Y2_GPIO_NUM;
  config.pin_d1 = Y3_GPIO_NUM;
  config.pin_d2 = Y4_GPIO_NUM;
  config.pin_d3 = Y5_GPIO_NUM;
  config.pin_d4 = Y6_GPIO_NUM;
  config.pin_d5 = Y7_GPIO_NUM;
  config.pin_d6 = Y8_GPIO_NUM;
  config.pin_d7 = Y9_GPIO_NUM;
  config.pin_xclk = XCLK_GPIO_NUM;
  config.pin_pclk = PCLK_GPIO_NUM;
  config.pin_vsync = VSYNC_GPIO_NUM;
  config.pin_href = HREF_GPIO_NUM;
  config.pin_sccb_sda = SIOD_GPIO_NUM;
  config.pin_sccb_scl = SIOC_GPIO_NUM;
  config.pin_pwdn = PWDN_GPIO_NUM;
  config.pin_reset = RESET_GPIO_NUM;
```

```
  config.xclk_freq_hz = 20000000;

  config.pixel_format = PIXFORMAT_JPEG;

  config.frame_size = FRAMESIZE_CIF;

  config.jpeg_quality = 16;

  config.fb_count = 3;
  config.fb_location = CAMERA_FB_IN_PSRAM;

  esp_camera_init(&config);

  httpd_handle_t stream_httpd = NULL;
  httpd_config_t server_config = HTTPD_DEFAULT_CONFIG();
  server_config.stack_size = 8192;

  httpd_uri_t stream_uri = {
     .uri       = "/stream",
     .method    = HTTP_GET,
     .handler   = stream_handler,
     .user_ctx  = NULL
  };

  if (httpd_start(&stream_httpd, &server_config) == ESP_OK) {
    httpd_register_uri_handler(stream_httpd, &stream_uri);
  }
}
```

```
void camera_server_task(void *pvParameters) {
  startCameraServer();
  vTaskDelete(NULL);
}
```